Computer Networks (Sessional)

Course: CSE-312

Explainable Malicious Node Detection in Wireless Sensor Networks

Leakage-Free Ensemble Learning with Class-Imbalance Handling, Class-Threshold Optimisation and SHAP/LIME Explanations on WSN-DS



Team Members

Name	Student ID
Al Rubayet	2204002
Md. Rony Hossain	2204005
Imteajul Islam	2204015
Tahsin Ahmed Chowdhury	2204028

Department of Computer Science & Engineering

Chittagong University of Engineering & Technology

Contents

1 Introduction **4**

2 Research Problems and Objectives **4**

2.1 Identified Research Problems 4

2.2 Objectives and Contributions 5

3 Related Work **5**

4 Dataset Overview **5**

4.1 Dataset Description 5

4.2 Dataset Glimpse 6

4.3 Class Distribution and Challenges 7

4.4 Final Dataset After Preprocessing 7

5 Methodology **7**

5.1 Overall Workflow 7

6 Data Preprocessing **8**

6.1 Missing Values and Column Cleaning 8

6.2 Removing Node Identifiers (Leakage Prevention) 8

6.3 Experiment: How Much Does Leakage Inflate Scores? 9

7 Exploratory Analysis and Feature Engineering **9**

7.1 Feature Behaviour per Class 9

7.2 Engineered Features 10

8 Feature Selection **10**

8.1 Ensemble Ranking Strategy 10

8.2 Choosing the Number of Features 11

9 Data Splitting and Class Balancing **12**

9.1 Stratified Train / Validation / Test Split 12

9.2 Class Balancing Strategies (Training Set Only) 12

10 Machine Learning Models **14**

10.1 Baseline Models 14

10.2 Hyper-parameter Tuning 14

10.3 Weighted Soft-Voting Ensemble 15

10.4 Class-wise Threshold Optimisation 15

11 Results and Analysis **15**

11.1 Baseline Model Performance 15

11.2 Improved Models 16
11.3 Improvement Summary 16
11.4 Confusion Matrix Analysis 17
12 Comparison with Recent Studies **19**
13 Model Explainability (SHAP and LIME) **20**
13.1 SHAP Methodology 20
13.2 Global Feature Importance 20
13.3 Feature Impact per Attack (Beeswarm) 21
13.4 Individual Prediction Explanation 22
13.5 LIME Local Explanation 23
14 Implementation and Reproducibility **23**
15 Conclusion and Future Work **24**
References **24**

1 Introduction

A **Wireless Sensor Network (WSN)** is a network of small, battery-powered sensor nodes that sense the environment (temperature, motion, pollution, etc.) and forward the readings over multi-hop wireless links to a **Base Station (BS)**. WSNs are used in smart agriculture, health monitoring, industrial IoT and military surveillance. To save energy, most WSNs use a hierarchical (cluster-based) routing protocol such as **LEACH** (Low-Energy Adaptive Clustering Hierarchy): in every round some nodes elect themselves **Cluster Heads (CH)**, the other nodes join a CH, the CH gives each member a TDMA time-slot, collects their data and forwards it to the BS.

Because the nodes are cheap, unattended and talk over an open radio channel, an attacker can easily capture or impersonate a node and turn it into a **malicious node**. A malicious node that participates in routing can silently destroy the network's service. These are **Denial-of-Service (DoS)** attacks at the network/routing layer, the main topic of this Computer Networks project:

- **Blackhole attack:** the malicious node advertises itself as a cluster head at the start of every round, attracts the traffic of its neighbours and then *drops all* received packets.
- **Grayhole attack:** same trick as blackhole, but the node drops packets *selectively or randomly*, which makes it much harder to detect.
- **Flooding attack:** the attacker sends a huge number of LEACH advertisement (ADV) messages, wasting the energy and bandwidth of the other nodes and confusing CH selection.
- **Scheduling (TDMA) attack:** a malicious CH gives *every* member the same TDMA slot, causing packet collisions and data loss.

Detecting these malicious nodes manually is impossible at scale, and classic signature-based intrusion detection does not fit tiny sensor nodes. In this project we build an **AI-based Intrusion Detection System (IDS)** that learns the behaviour of normal and malicious nodes from 374,661 simulated node records (the public WSN-DS dataset) and classifies each node into one of five classes. Beyond accuracy, we focus on honest evaluation (no data leakage), fair metrics for the rare attack classes, and explainability, so that a network operator can see *why* a node was flagged.

2 Research Problems and Objectives

2.1 Identified Research Problems

Reviewing the literature on WSN intrusion detection and inspecting the WSN-DS data, we identified the following open problems:

- **RP1: Severe class imbalance.** Normal records make up 90.8% of the data, while Flooding is only 0.88% (imbalance ratio 103:1). A model that predicts 'Normal' for everything already reaches 90.8% accuracy, so *accuracy alone is misleading*; rare attacks are missed.
- **RP2: Blackhole vs Grayhole confusion.** Both attacks behave identically during CH election and differ only in how many packets they drop. The original WSN-DS paper reports only 75.6% detection for Grayhole.

- **RP3: Optimistic evaluation / data leakage.** Many published works keep the node identifier columns (*id*, *who_CH*) and duplicate rows. The model can then memorise *which* simulated node was malicious instead of learning *how* a malicious node behaves, and duplicated records can appear in both train and test sets. Both make reported scores optimistic and do not transfer to a real network.
- **RP4: Black-box models.** Ensemble models are accurate but opaque. Network operators need to know which traffic/energy behaviour triggered an alarm before isolating a node.

2.2 Objectives and Contributions

- Build a **leakage-free pipeline** (identifier removal, de-duplication before splitting, untouched test set).
- Engineer **routing-behaviour features** (packet forwarding ratio, control-message volume, energy per packet).
- Apply **ensemble feature selection** (Mutual Information + Chi-square + Random-Forest importance).
- Handle imbalance by **comparing four balancing strategies** (none, class weights, SMOTE, under-sampling + SMOTE) on the training data only.
- Compare **7 baseline classifiers**, tune the best ones with **RandomizedSearchCV**, build a weighted **soft-voting ensemble** and apply **class-wise threshold optimisation** to maximise macro-F1.
- Explain the model with **SHAP** (global + local) and **LIME**, and compare against recent studies.

3 Related Work

Almomani et al. (2016) created the WSN-DS dataset by simulating LEACH in NS-2 and trained an Artificial Neural Network, reporting per-class detection of 92.8% (Blackhole), 75.6% (Grayhole), 99.4% (Flooding), 92.2% (Scheduling) and 99.8% (Normal) [1]. **Talukder et al. (2024)** proposed MLSTL-WSN, which balances WSN-DS with SMOTE-Tomek and evaluates several machine-learning classifiers, reporting 99.78% binary and 99.92% multiclass accuracy [2]. **Ouhmi et al. (2026)** compared six boosting algorithms (AdaBoost, GB, XGBoost, LightGBM, CatBoost, HistGB) for WSN intrusion detection on WSN-DS and reported 99.18% accuracy with a 12.7 ms detection time [3]. Most of these works report accuracy as the headline metric and do not discuss the identifier columns, which motivates our RP1 and RP3.

4 Dataset Overview

4.1 Dataset Description

We use **WSN-DS** [1], a public intrusion-detection dataset for wireless sensor networks. It was generated with the NS-2 simulator running the LEACH routing protocol; each record describes the state of one sensor node in one round. The raw file contains **374,661 records** and **19 columns** (18 features + 1 label), with **0 missing values**.

#	Feature	Description	Type
1	id	Unique node ID (encodes round + node number)	Identifier
2	Time	Simulation time of the node record	Numeric
3	Is_CH	1 if the node is a cluster head in this round, else 0	Binary
4	who CH	ID of the cluster head the node belongs to	Identifier
5	Dist_To_CH	Distance between the node and its CH	Numeric
6	ADV_S	Number of CH advertisement messages sent	Count
7	ADV_R	Number of CH advertisement messages received	Count
8	JOIN_S	Number of join-request messages sent	Count
9	JOIN_R	Number of join-request messages received (by a CH)	Count
10	SCH_S	Number of TDMA schedule messages sent (by a CH)	Count
11	SCH_R	Number of TDMA schedule messages received	Count
12	Rank	Order of the node in the TDMA schedule	Numeric
13	DATA_S	Number of data packets sent by the node to its CH	Count
14	DATA_R	Number of data packets received (by a CH)	Count
15	Data_Sent_To_BS	Number of data packets the CH sent to the base station	Count
16	dist_CH_To_BS	Distance between the CH and the base station	Numeric
17	send_code	Cluster sending code	Numeric
18	Expanded Energy	Energy consumed by the node in the round	Numeric
19	Attack type	Target: Normal, Blackhole, Grayhole, Flooding, TDMA	Label

Table 1: Features of the WSN-DS dataset

4.2 Dataset Glimpse

Feature	Normal	Flooding	TDMA	Grayhole	Blackhole
id	101000	101096	106040	103003	102041
Time	50	53	303	153	103
Is_CH	1	1	1	1	1
who CH	101000	101096	106040	103003	102041
Dist_To_CH	0	0	0	0	0
ADV_S	1	7	1	1	1
ADV_R	0	0	1	4	3
JOIN_S	0	0	0	0	0
JOIN_R	25	90	89	22	93
SCH_S	1	1	89	1	1
SCH_R	0	0	0	0	0
Rank	0	0	0	0	0
DATA_S	0	0	0	0	0
DATA_R	1200	1350	0	1166	1302
Data_Sent_To_BS	48	15	0	29	0
dist_CH_To_BS	130.085	121.695	0	85.1979	0
send_code	0	0	0	0	0
Expaned Energy	2.4694	2.25865	0.00724	2.06959	0.00726

Table 2: Dataset glimpse: one raw record from each class (transposed: each column is one node record)

Each column shows the counters of one node in one LEACH round. Compare the columns: the Normal cluster head forwards part of what it receives to the base station, while attack records differ

in the advertisement (ADV_S), schedule (SCH_S) and forwarding counters.

4.3 Class Distribution and Challenges

Class	Meaning	Records	Share
Normal	Benign node	340,066	90.77%
Grayhole	Selective packet dropping	14,596	3.90%
Blackhole	Drops all packets	10,049	2.68%
TDMA	Scheduling attack (slot collision)	6,638	1.77%
Flooding	ADV message flooding	3,312	0.88%
Total		374,661	100%

Table 3: Class distribution of WSN-DS

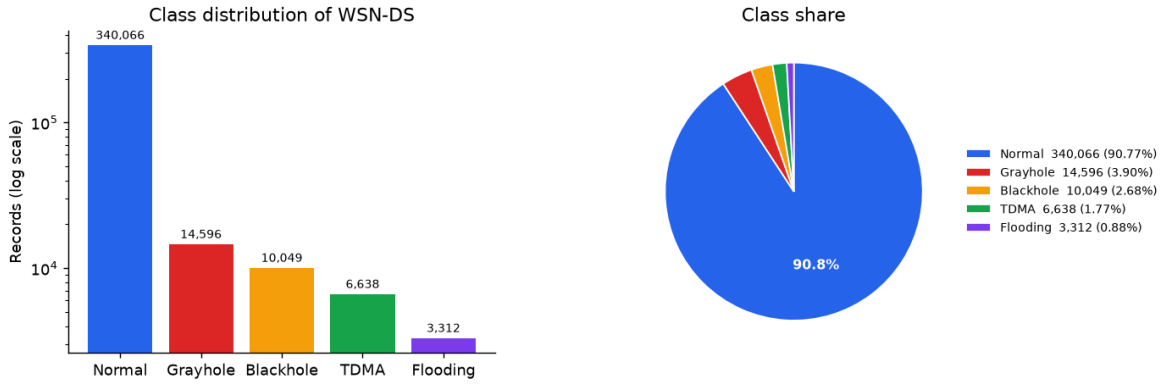


Figure 1: Class distribution of WSN-DS (left: log scale; right: share)

- **Extreme imbalance**: Normal:Flooding = 103:1.
- **Duplicates**: 8,873 exact duplicate rows; 23,250 duplicates after removing identifier columns.
- **Identifier columns** (*id*, *who CH*) that can leak the label.
- **Similar attacks**: Blackhole and Grayhole share the same CH-election behaviour.

4.4 Final Dataset After Preprocessing

Item	Value
Records after cleaning	351,411
Features used (16 original + 3 engineered)	19
Classes	Normal (324,062), Grayhole (12,060), TDMA (6,495), Blackhole (5,637), Flooding (3,157)
Train / Validation / Test	253,015 / 28,113 / 70,283

Table 4: Dataset summary after preprocessing

5 Methodology

5.1 Overall Workflow

Figure 2 shows the complete pipeline. Every data-dependent step (feature selection, resampling, tuning, threshold optimisation) is fitted on the training or validation data only; the 20% test set is never touched until the final evaluation and keeps the real-world class imbalance.

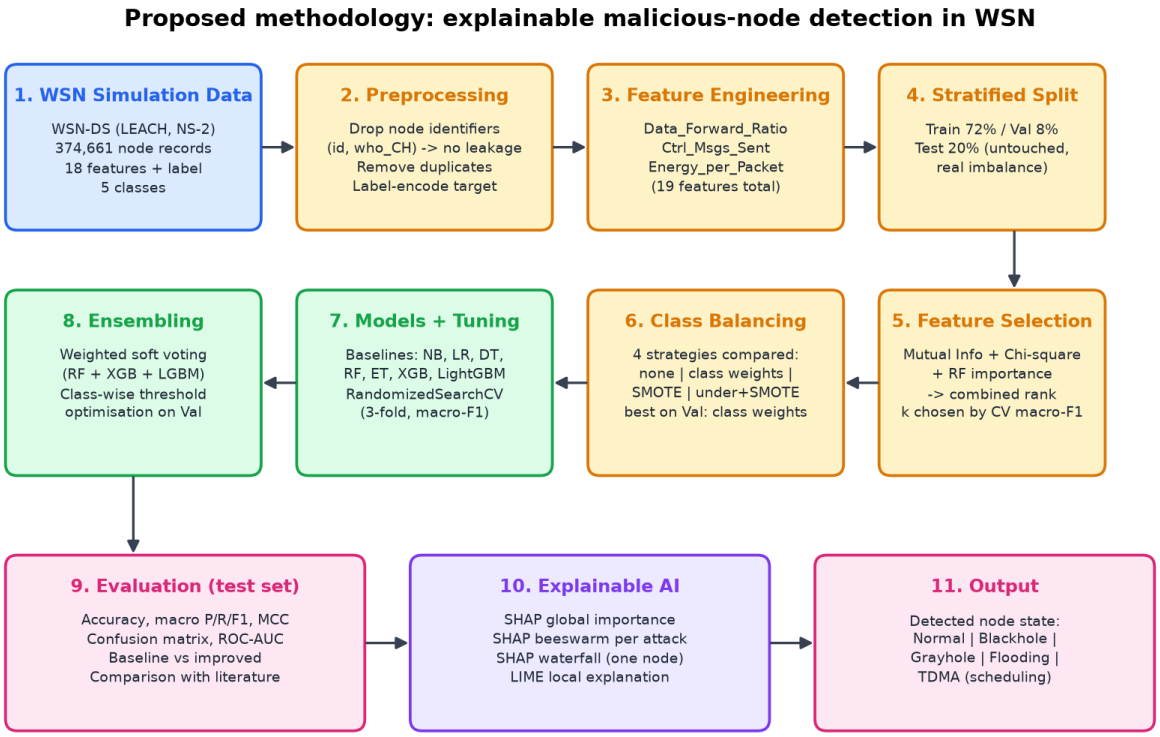


Figure 2: Proposed methodology workflow

6 Data Preprocessing

6.1 Missing Values and Column Cleaning

The dataset has no missing values, so no imputation is needed. Column names contain stray spaces (e.g. ' Time'), which are stripped and two names are corrected.

PYTHON | Listing 1: Loading and cleaning column names

```python
df = pd.read_csv("data/WSN-DS.csv")
df.columns = [c.strip() for c in df.columns]
df = df.rename(columns={"Expaned Energy": "Expanded_Energy",
                        "who CH": "Who_CH", "Attack type": "Attack_Type"})
print(df.isna().sum().sum())          # -> 0 missing values
```

6.2 Removing Node Identifiers (Leakage Prevention)

The *id* column encodes the simulation round and node number, and *who_CH* stores the ID of the cluster head. In a simulation, the same few node IDs are configured as attackers, so these columns let the model memorise attacker IDs. A real IDS must recognise malicious *behaviour* of unseen nodes, so both columns are dropped.

PYTHON | Listing 2: Identifier removal and de-duplication

```python
df = df.drop(columns=["id", "Who_CH"])          # identifiers -> leakage
df = df.drop_duplicates().reset_index(drop=True) # before splitting!
```

Result: 23,250 duplicate records removed; 351,411 unique records remain. De-duplicating *before* the train/test split guarantees that no identical record appears in both sets.

6.3 Experiment: How Much Does Leakage Inflate Scores?

To test RP3 we trained default LightGBM and Random Forest models twice: once with identifiers and duplicates kept (as commonly done) and once with our cleaning (80/20 stratified split in both cases).

Setting	Model	Accuracy	Macro F1	Top-3 important features	Rank of id / who_CH
IDs + duplicates kept	LightGBM	0.9923	0.9557	Expaned Energy, id, who CH	2, 3
IDs + duplicates kept	Random Forest	0.9973	0.9819	ADV_S, Expaned Energy, Is_CH	13, 14
IDs + duplicates removed (ours)	LightGBM	0.9960	0.9727	Time, Expaned Energy, ADV_R	-
IDs + duplicates removed (ours)	Random Forest	0.9965	0.9750	ADV_S, Is_CH, Expaned Energy	-

Table 5: Effect of identifier columns and duplicates (default models)

Two effects are visible. (1) With duplicates kept, Random Forest's macro-F1 is +0.69 pp higher, because identical records end up in both train and test sets and are simply memorised. (2) LightGBM ranks the *id* and *who_CH* columns 2nd and 3rd, so it partly learns *which* node was the attacker; this does not help on this split (its macro-F1 is actually lower) and cannot transfer to a real network with new node IDs. Both effects are avoided by our stricter setting, so all results below are leakage-free.

7 Exploratory Analysis and Feature Engineering

7.1 Feature Behaviour per Class

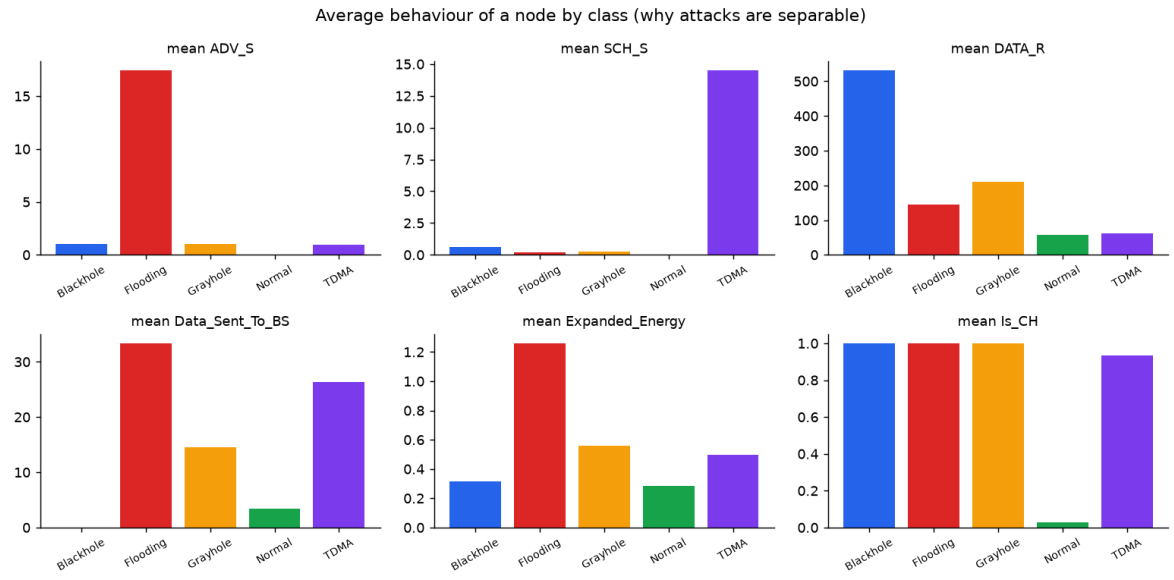


Figure 3: Average value of key features per class

The class means already show the network-level signatures of each attack: Flooding nodes send far more advertisement messages (ADV_S); TDMA attackers send many schedule messages (SCH_S); Blackhole and Grayhole nodes are almost always cluster heads (Is_CH) that receive a lot of data (DATA_R) but forward little or nothing to the base station (Data_Sent_To_BS).

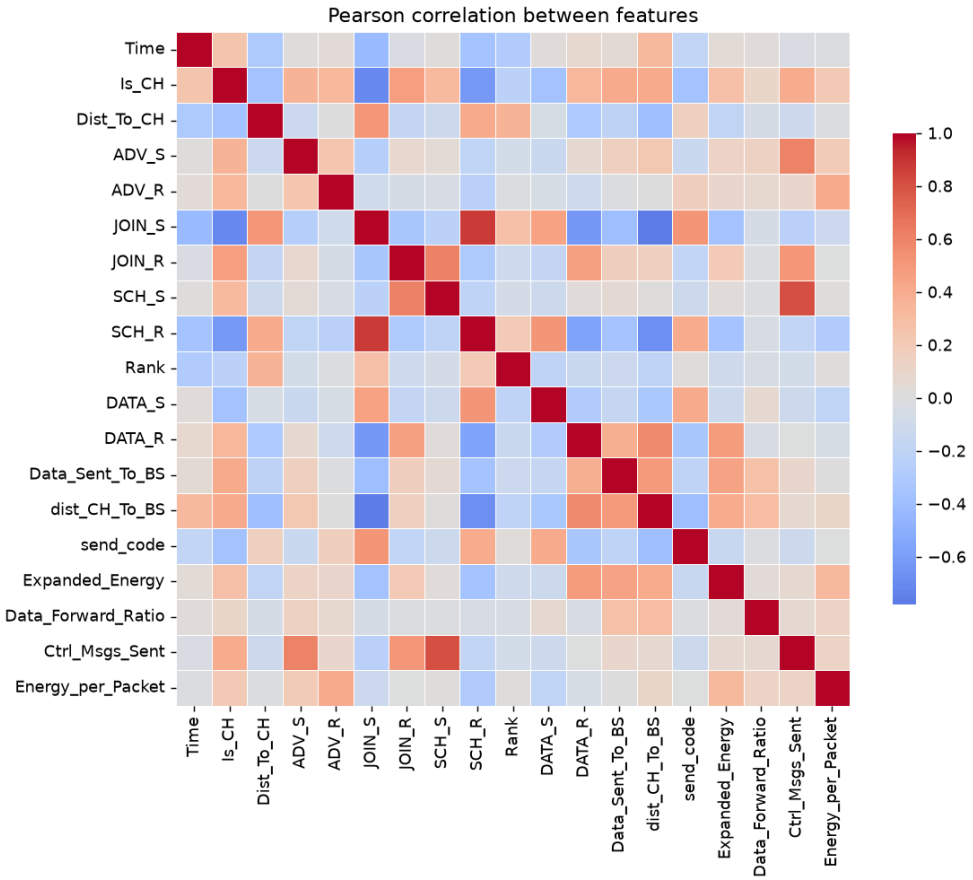


Figure 4: Pearson correlation matrix of all features

7.2 Engineered Features

Based on how each attack works at the routing layer, three behaviour features were added:

PYTHON | Listing 3: Routing-behaviour feature engineering

```python
# Does a cluster head forward what it receives?  (Blackhole ~ 0, Grayhole < 1)
df["Data_Forward_Ratio"] = df["Data_Sent_To_BS"] / (df["DATA_R"] + 1)
# Control-plane chatter (Flooding / TDMA attacks send many control messages)
df["Ctrl_Msgs_Sent"]     = df["ADV_S"] + df["JOIN_S"] + df["SCH_S"]
# Energy cost per handled packet
df["Energy_per_Packet"]  = df["Expanded_Energy"] / (df["DATA_S"] + df["DATA_R"] + 1)
```

8 Feature Selection

8.1 Ensemble Ranking Strategy

Three complementary scores are computed on a 60,000-record sample of the **training set** and min-max normalised: Mutual Information (non-linear dependency), Chi-square (statistical dependency on min-max scaled features) and Random-Forest impurity importance (model-based). They are combined as **0.4 MI + 0.2 Chi2 + 0.4 RF**.

PYTHON | Listing 4: Ensemble feature scoring

```python
mi  = mutual_info_classif(Xs, ys, random_state=42)
ch, _ = chi2(MinMaxScaler().fit_transform(Xs), ys)
rf  = RandomForestClassifier(150, class_weight="balanced").fit(Xs, ys).feature_importances_
combined = 0.4*norm(mi) + 0.2*norm(ch) + 0.4*norm(rf)
```

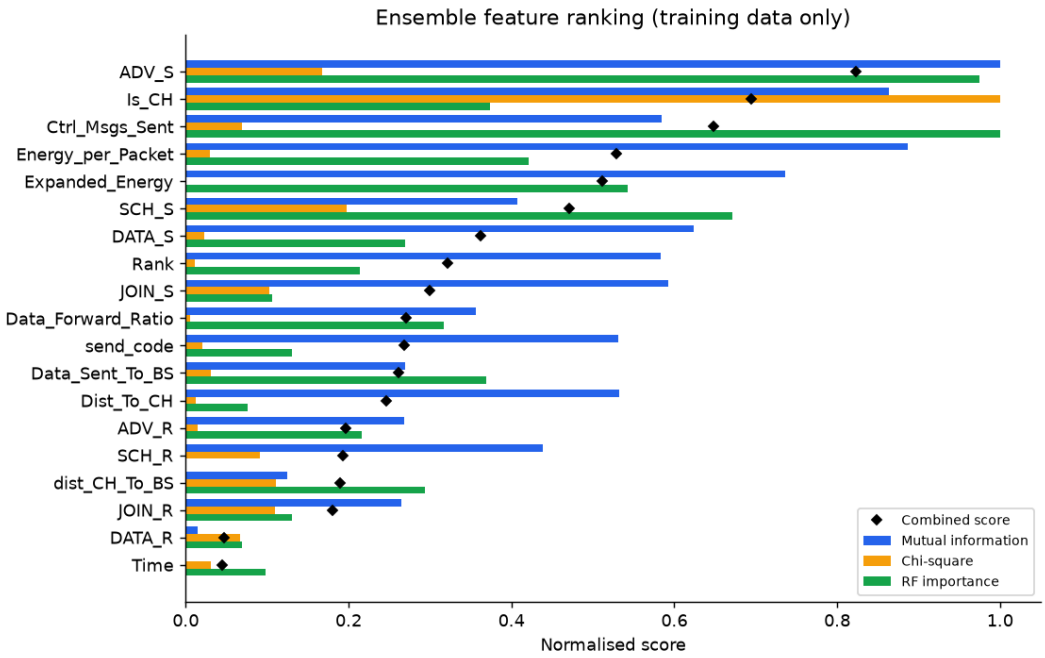


Figure 5: Normalised feature scores (sorted by combined score)

Rank	Feature	MI	Chi2	RF	Combined
1	ADV_S	1.000	0.168	0.974	0.823
2	Is_CH	0.864	1.000	0.373	0.695
3	Ctrl_Msgs_Sent	0.585	0.069	1.000	0.648
4	Energy_per_Packet	0.887	0.030	0.422	0.529
5	Expanded_Energy	0.737	0.000	0.542	0.512
6	SCH_S	0.407	0.198	0.672	0.471
7	DATA_S	0.624	0.023	0.269	0.362
8	Rank	0.584	0.011	0.214	0.321
9	JOIN_S	0.593	0.102	0.107	0.300
10	Data_Forward_Ratio	0.356	0.005	0.317	0.270

Table 6: Top-10 features by combined score

8.2 Choosing the Number of Features

The top-*k* features were evaluated for k in {4, 6, 8, 10, 12, 14, 19} with 3-fold cross-validated macro-F1 (LightGBM). The smallest k whose score is within 0.2 percentage points of the best was chosen.

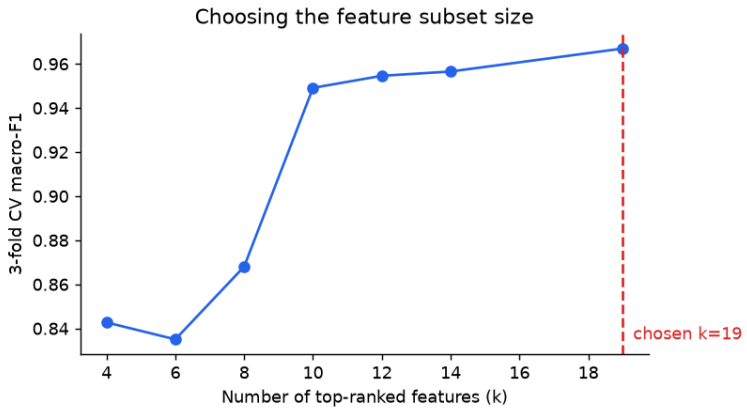


Figure 6: Cross-validated macro-F1 versus number of selected features

Selected: k = 19 features. Every reduction of the feature set lowered macro-F1, so all features were kept for the main model; the ranking is still useful for a lightweight on-node deployment (the top-12 features reach 0.955 macro-F1 versus 0.967 with all).

9 Data Splitting and Class Balancing

9.1 Stratified Train / Validation / Test Split

PYTHON | Listing 5: Stratified three-way split

```
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.20, stratify=y, ran
dom_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.10,
                                                  stratify=y_tmp, random_state=42)
```

Train: 253,015 records (72%), validation: 28,113 (8%, used for ensemble weights and threshold optimisation), test: 70,283 (20%, original imbalance).

9.2 Class Balancing Strategies (Training Set Only)

Four ways of handling the imbalance were compared, all applied to the training data only:

- **No resampling**: train on the original distribution.
- **Class weights**: keep the data, but give every record of class c the weight sqrt(N_{max} / N_c), so mistakes on rare attacks cost more.
- **Moderate SMOTE**: synthesise new attack records until every attack class has 15,000 records.
- **Hybrid**: random **under-sampling** of Normal to 60,000 records plus **SMOTE** of every attack class to 20,000 records (also used to build the balanced sample for hyper-parameter tuning).

PYTHON | Listing 6: Class weighting and hybrid resampling

```
# Class weights (chosen strategy)
w = {c: (counts.max() / counts[c]) ** 0.5 for c in classes}
model.fit(X_train, y_train, sample_weight=[w[c] for c in y_train])
# Hybrid resampling (compared strategy)
X_rs, y_rs = RandomUnderSampler(sampling_strategy={normal: 60000}).fit_resample(X_tra
in, y_train)
X_rs, y_rs = SMOTE(sampling_strategy={c: 20000 for c in attacks}).fit_resample(X_rs,
y_rs)
```

Class	Train records	Class weight	After hybrid resampling
Blackhole	4,058	7.58	20,000
Flooding	2,274	10.13	20,000
Grayhole	8,683	5.18	20,000
Normal	233,324	1.00	60,000
TDMA	4,676	7.06	20,000
Total	253,015		140,000

Table 7: Training-set class counts, class weights and hybrid-resampled counts

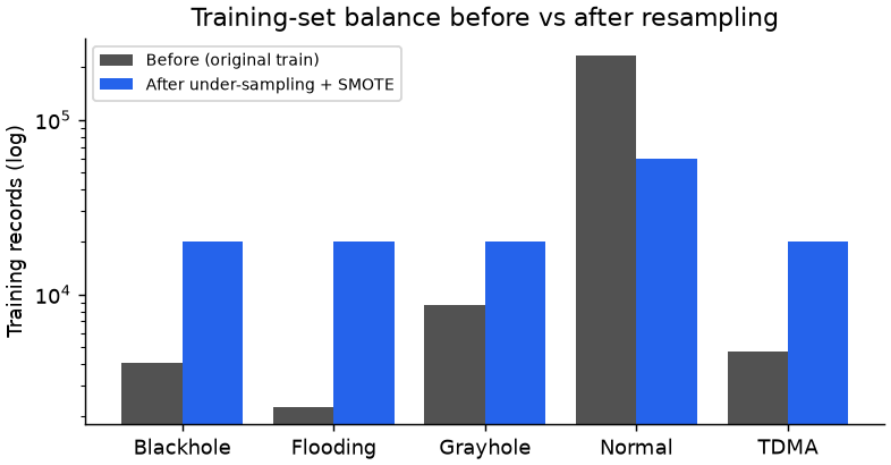


Figure 7: Training class distribution before and after hybrid resampling

Strategy	Variant	Val macro-F1	Test precision	Test recall	Test macro-F1
No resampling	ensemble	0.9773	0.9811	0.9753	0.9779
No resampling	ensemble + threshold opt.	0.9788	0.9784	0.9803	0.9789
Class weights (sqrt-inverse)	ensemble	0.9771	0.9791	0.9781	0.9783
Class weights (sqrt-inverse)	ensemble + threshold opt.	0.9791	0.9801	0.9778	0.9786
Moderate SMOTE (minorities to 15k)	ensemble	0.9766	0.9765	0.9791	0.9775
Moderate SMOTE (minorities to 15k)	ensemble + threshold opt.	0.9786	0.9795	0.9786	0.9787
Hybrid under-sampling + SMOTE	ensemble	0.9673	0.9540	0.9815	0.9673
Hybrid under-sampling + SMOTE	ensemble + threshold opt.	0.9740	0.9711	0.9814	0.9757

Table 8: Balancing ablation with the tuned RF+XGBoost+LightGBM soft-voting ensemble

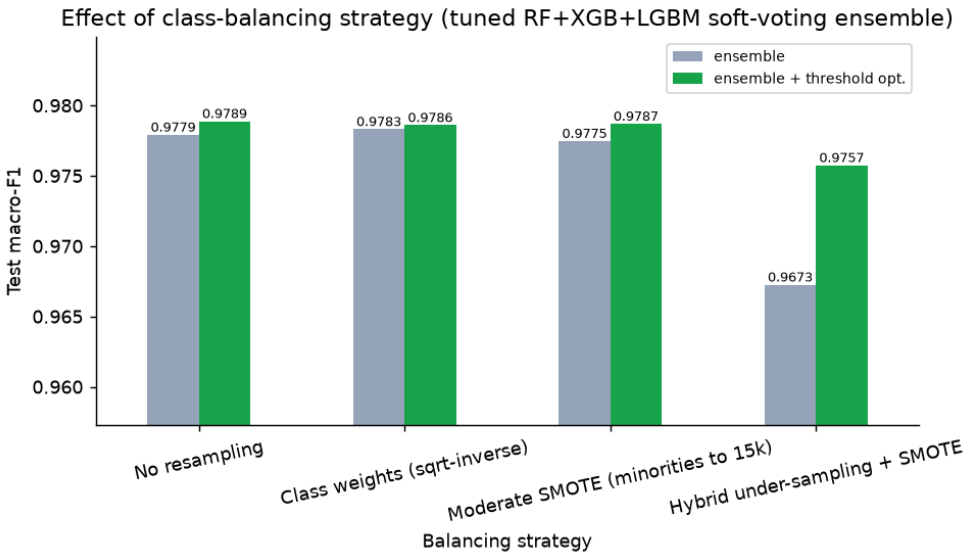


Figure 8: Test macro-F1 for each balancing strategy, with and without threshold optimisation

The strategy was chosen by **validation** macro-F1, not by test score: **Class weights (sqrt-inverse)**. A key finding is that heavy resampling (hybrid) gives the highest recall but clearly the lowest precision: SMOTE creates synthetic attack records inside the dense Normal region, so the model raises false alarms on normal nodes. Mild class weighting keeps precision high while still emphasising rare attacks. Threshold optimisation improves every strategy, and helps the hybrid one most because it repairs the precision loss.

10 Machine Learning Models

10.1 Baseline Models

Seven classifiers were first trained on the original (imbalanced) training data with default parameters: Gaussian Naive Bayes, Logistic Regression, Decision Tree, Random Forest, Extra Trees, XGBoost and LightGBM. These represent probabilistic, linear, single-tree, bagging and boosting families.

10.2 Hyper-parameter Tuning

The three strongest families (Random Forest, XGBoost, LightGBM) were tuned with **RandomizedSearchCV** (12 random configurations, 3-fold stratified CV, **macro-F1** scoring) on a 40,000-record sample of the balanced training set.

PYTHON | Listing 7: Randomized hyper-parameter search (LightGBM)

```
space = {"n_estimators": [300, 500, 800], "num_leaves": [31, 63, 127, 255],
         "learning_rate": [0.03, 0.05, 0.1], "max_depth": [-1, 10, 16],
         "min_child_samples": [10, 20, 40], "colsample_bytree": [0.7, 0.85, 1.0], ...
}
search = RandomizedSearchCV(LGBMClassifier(), space, n_iter=12, cv=3, scoring="f1_mac
ro")
search.fit(X_bal_sample, y_bal_sample)
```

Model	Best CV macro-F1	Mean CV macro-F1	Best parameters
Random Forest	0.9842	0.9834	n_estimators=400, min_samples_split=2, min_samples_leaf=1, max_features=0.5, max_depth=40
XGBoost	0.9857	0.9853	subsample=0.8, n_estimators=700, min_child_weight=1, max_depth=6, learning_rate=0.2, colsample_bytree=0.7
LightGBM	0.9861	0.9858	subsample_freq=1, subsample=0.8, reg_lambda=0, num_leaves=127, n_estimators=800, min_child_samples=40, max_depth=-1, learning_rate=0.1, colsample_bytree=0.85

Table 9: Hyper-parameter tuning results

10.3 Weighted Soft-Voting Ensemble

The tuned RF, XGBoost and LightGBM models are combined by averaging their class probabilities, weighted by each model's validation macro-F1 (RF (tuned+cw): 0.332, XGBoost (tuned+cw): 0.334, LightGBM (tuned+cw): 0.334).

10.4 Class-wise Threshold Optimisation

A classifier predicts the class with the highest probability. With imbalanced costs this is not optimal for macro-F1. We learn one multiplicative weight per class on the **validation set** using coordinate ascent (grid 0.5-3.0) and predict argmax($w_c \cdot P(c|x)$).

PYTHON | Listing 8: Class-wise threshold (weight) optimisation

```python
def optimise_class_weights(P, y, grid=np.linspace(0.5, 3.0, 26)):
    w = np.ones(P.shape[1]); best = f1_score(y, P.argmax(1), average="macro")
    for _ in range(3):
        for c in range(P.shape[1]):
            for g in grid:
                cand = w.copy(); cand[c] = g
                f = f1_score(y, (P*cand).argmax(1), average="macro")
                if f > best: best, w = f, cand
    return w
```

It was applied to the best validation model (**LightGBM (tuned+cw)**). Learned weights: Blackhole = 1.0, Flooding = 1.0, Grayhole = 0.8, Normal = 1.0, TDMA = 1.0.

11 Results and Analysis

All numbers below are measured on the untouched, imbalanced test set (70,283 records). Because of RP1 we treat **macro-F1** (the unweighted mean of the five class F1-scores) and **MCC** as the main metrics; accuracy is reported for completeness.

11.1 Baseline Model Performance

Model	Accuracy	Precision	Recall	Macro F1	MCC	Train (s)
Random Forest	0.9964	0.9751	0.9748	0.9745	0.9756	6.7
Extra Trees	0.9963	0.9736	0.9743	0.9736	0.9748	5.3
LightGBM	0.9961	0.9743	0.9724	0.9732	0.9733	3.1
XGBoost	0.9957	0.9721	0.9630	0.9673	0.9710	4.5
Decision Tree	0.9941	0.9587	0.9654	0.9620	0.9599	2.5
Logistic Regression	0.9882	0.9377	0.8920	0.9110	0.9190	7.3
Gaussian NB	0.9583	0.7177	0.8692	0.7539	0.7621	0.5

Table 10: Baseline results (imbalanced training, default parameters, macro-averaged)

Tree ensembles clearly outperform linear and probabilistic models. Note how Gaussian NB reaches 95.83% accuracy but only 0.754 macro-F1: high accuracy hides poor detection of rare attacks, which is exactly RP1.

11.2 Improved Models

Model	Accuracy	Precision	Recall	Macro F1	Weighted F1	MCC
LightGBM (tuned+cw) + Threshold Opt.	0.9970	0.9806	0.9782	0.9791	0.9970	0.9797
LightGBM (tuned+cw)	0.9970	0.9804	0.9778	0.9788	0.9970	0.9795
Soft Voting Ensemble	0.9969	0.9791	0.9781	0.9783	0.9969	0.9788
XGBoost (tuned+cw)	0.9967	0.9773	0.9780	0.9774	0.9967	0.9778
RF (tuned+cw)	0.9963	0.9721	0.9778	0.9745	0.9963	0.9749

Table 11: Results after feature engineering, tuning, class weighting (cw), ensembling and threshold optimisation

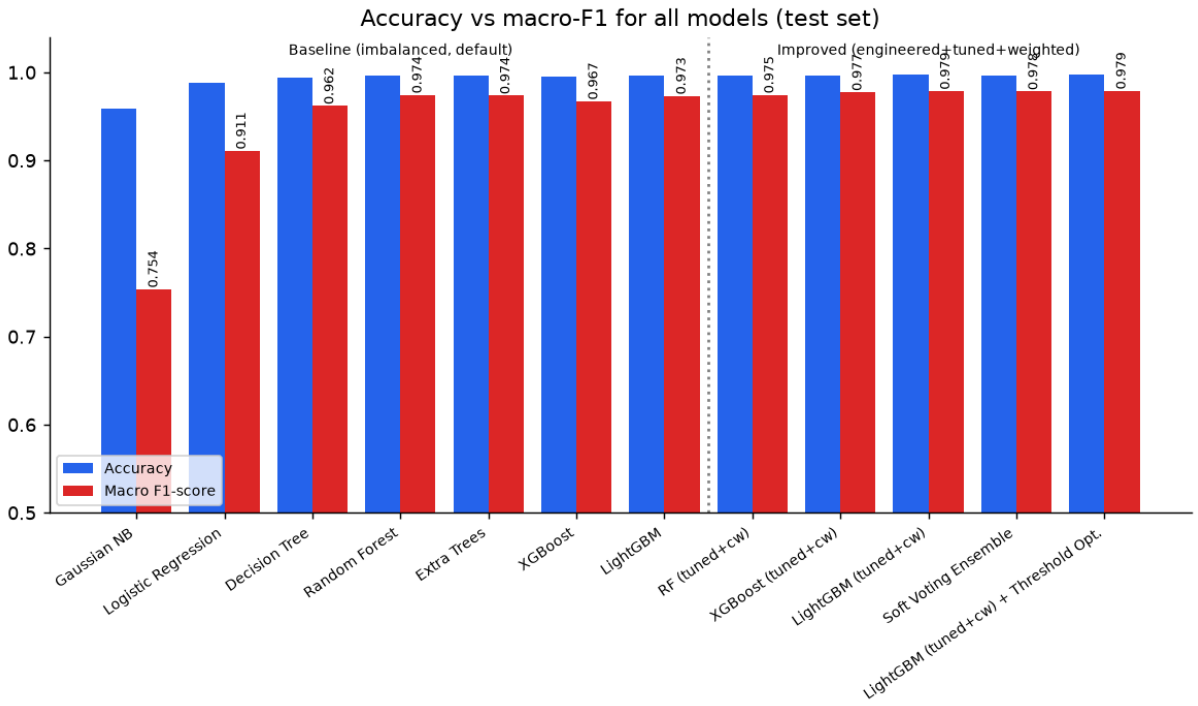


Figure 9: Accuracy and macro-F1 of all baseline and improved models

11.3 Improvement Summary

Metric	Best baseline (Random Forest)	Final (LightGBM (tuned+cw) + Threshold Opt.)	Change
Accuracy	0.9964	0.9970	+0.06 pp
Precision (macro)	0.9751	0.9806	+0.55 pp
Recall (macro)	0.9748	0.9782	+0.34 pp
F1 (macro)	0.9745	0.9791	+0.46 pp
MCC	0.9756	0.9797	+0.41 pp

Table 12: Best baseline versus final model (pp = percentage points)

The final model reaches **99.70% accuracy** and **97.91% macro-F1** (+0.46 pp macro-F1 over the best baseline). Mild class weighting raises the *recall* of rare attack classes without the false alarms caused by heavy SMOTE, and threshold optimisation then fine-tunes the precision/recall balance of each class, giving the best trade-off.

11.4 Confusion Matrix Analysis

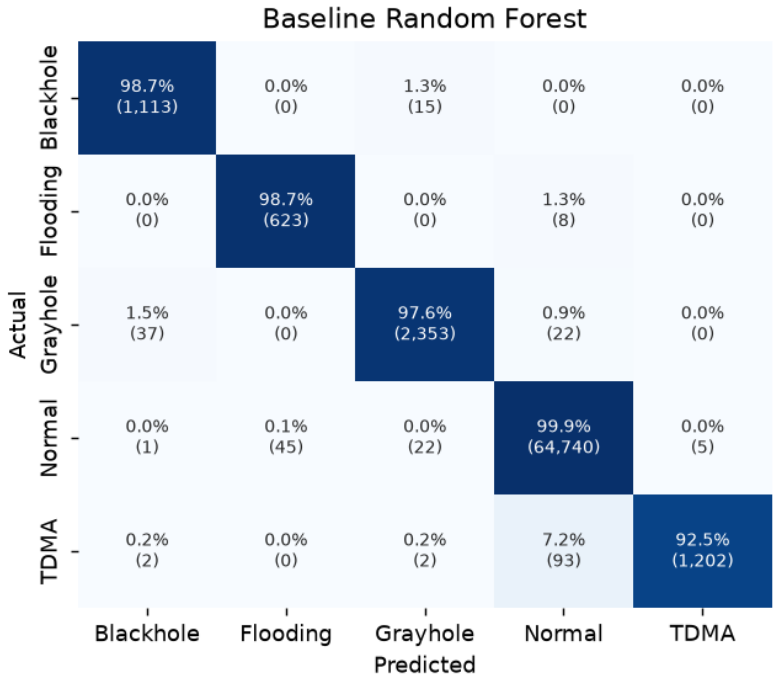


Figure 10: Confusion matrix of the best baseline (Random Forest); rows = actual, row-normalised

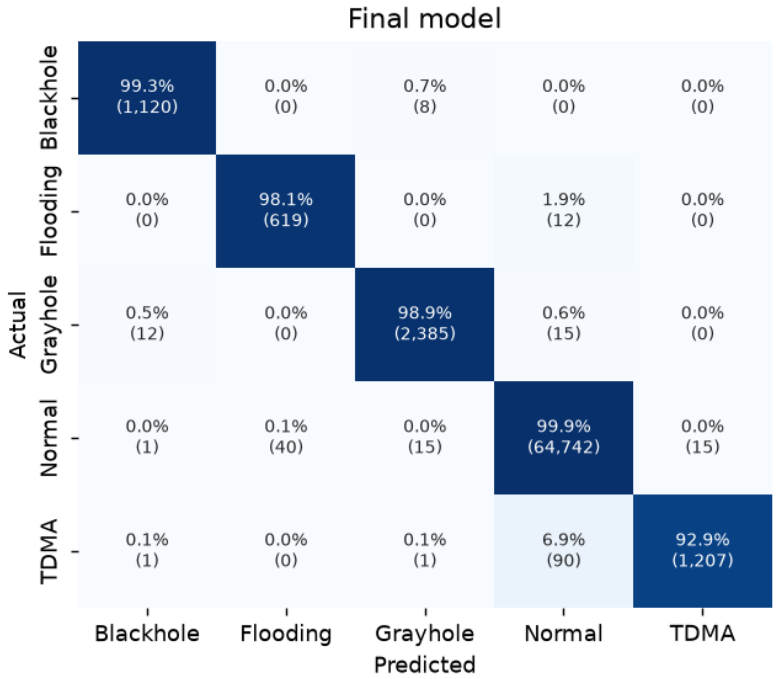


Figure 11: Confusion matrix of the final model (LightGBM (tuned+cw) + Threshold Opt.)

Class	Precision	Recall (detection rate)	F1-score	Support
Blackhole	0.9877	0.9929	0.9903	1,128
Flooding	0.9393	0.9810	0.9597	631
Grayhole	0.9900	0.9888	0.9894	2,412
Normal	0.9982	0.9989	0.9986	64,813
TDMA	0.9877	0.9292	0.9576	1,299
Macro avg	0.9806	0.9782	0.9791	70,283

Table 13: Per-class performance of the final model

The hardest class is **TDMA** (F1 = 0.9576). The three largest error types are: TDMA predicted as Normal (90 nodes); Normal predicted as Flooding (40 nodes); Normal predicted as TDMA (15 nodes). TDMA attackers that are confused with Normal are nodes that act as ordinary members in a round, so their own counters look normal; Blackhole/Grayhole confusion is expected because both attacks behave identically during cluster-head election and differ only in the fraction of packets dropped (RP2).

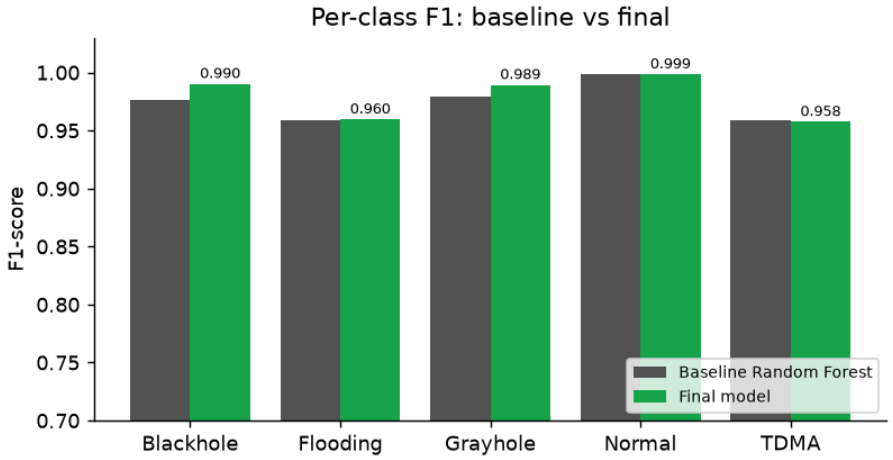


Figure 12: Per-class F1-score of best baseline versus final model

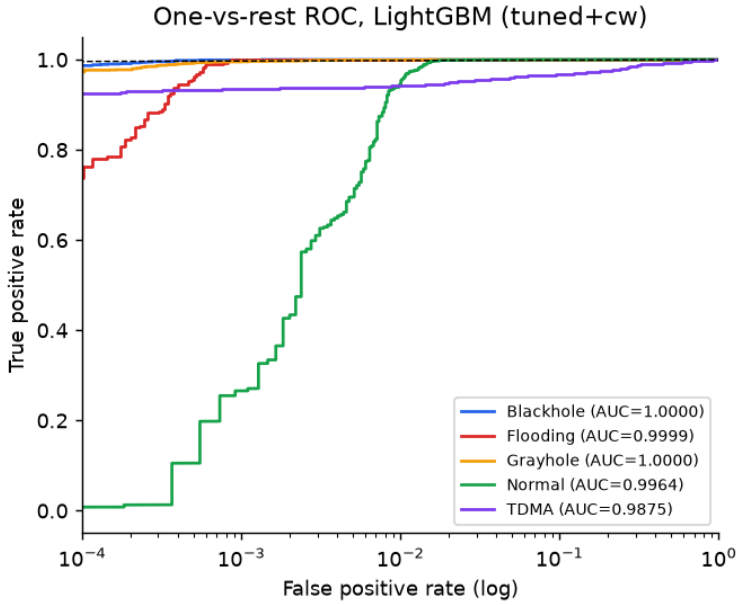


Figure 13: One-vs-rest ROC curves (log-scaled FPR)

12 Comparison with Recent Studies

Study	Method	Evaluation setting	Accuracy	Macro F1
Almomani et al. [1] (2016)	Artificial Neural Network	IDs kept	Per-class only	n/r
Talukder et al. [2] (2024)	SMOTE-Tomek + ML (MLSTL-WSN)	IDs/duplicates not discussed	99.92%	n/r
Ouhmi et al. [3] (2026)	Boosting ensemble (6 algorithms)	Not discussed	99.18%	n/r
This work	**LightGBM (tuned+cw) + Threshold Opt.**	**Leakage-free, de-duplicated**	**99.70%**	**97.91%**

Table 14: Overall comparison with recent studies on WSN-DS (n/r = not reported)

Class	Almomani et al. [1] ANN	This work (recall)	Gain
Blackhole	92.80%	99.29%	+6.49 pp
Flooding	99.40%	98.10%	-1.30 pp
Grayhole	75.60%	98.88%	+23.28 pp
Normal	99.80%	99.89%	+0.09 pp
TDMA	92.20%	92.92%	+0.72 pp

Table 15: Per-class detection rate compared with the original WSN-DS study

Compared with the original ANN, our model detects more attackers in 4 of 5 classes (Blackhole, Grayhole, Normal, TDMA), with the largest gain on the hard Grayhole class (+23.3 pp, RP2); it is slightly lower for Flooding (-1.30 pp), partly because our test set has no duplicate records to memorise. The 99%+ accuracies of [2] and [3] are close to ours; however, those studies report accuracy (dominated by the Normal class) and do not state that identifier columns and duplicates were removed. Our leakage experiment (Table 5) shows that keeping duplicates can inflate scores, so the numbers are not strictly comparable; our evaluation is the more conservative one.

13 Model Explainability (SHAP and LIME)

13.1 SHAP Methodology

SHAP (SHapley Additive exPlanations) assigns each feature a contribution to a prediction based on Shapley values from cooperative game theory. We use **TreeExplainer** on the tuned LightGBM model with 2,000 random test records; for 5 classes this yields a 2000 x features x 5 array of contributions.

PYTHON | Listing 9: SHAP analysis

```python
explainer   = shap.TreeExplainer(lgbm)
shap_values = explainer.shap_values(X_test_sample)     # (2000, n_features, 5)
shap.summary_plot(shap_values[:, :, grayhole], X_test_sample)
```

13.2 Global Feature Importance

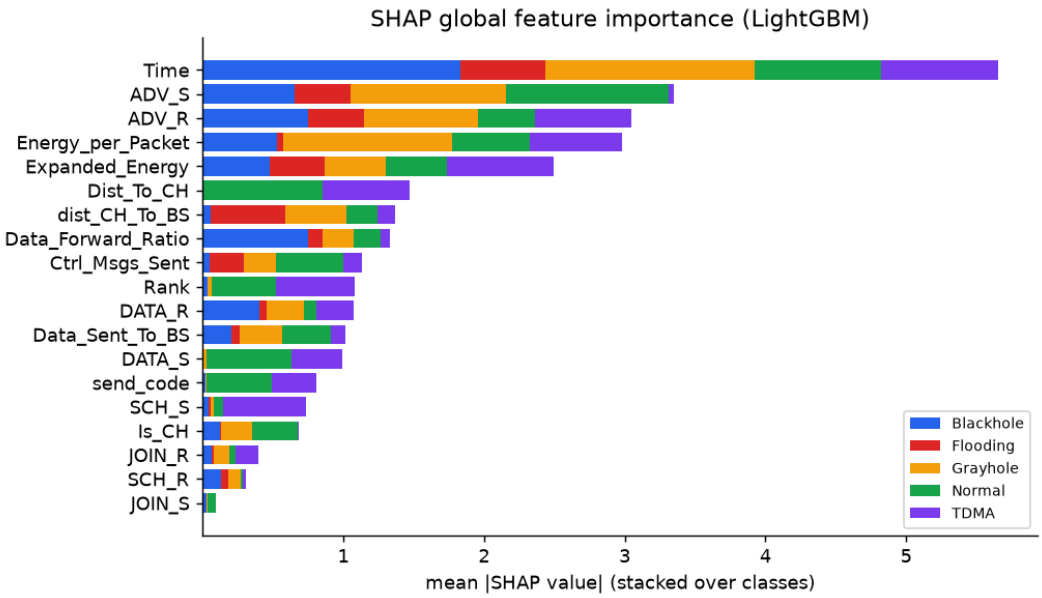


Figure 14: Mean |SHAP| per feature, stacked by class

Rank	Feature	Blackhole	Flooding	Grayhole	Normal	TDMA	Total
1	Time	1.835	0.597	1.489	0.897	0.835	5.654
2	ADV_S	0.652	0.402	1.102	1.157	0.035	3.348
3	ADV_R	0.749	0.401	0.807	0.408	0.684	3.050
4	Energy_per_Packet	0.525	0.050	1.196	0.556	0.651	2.978
5	Expanded_Energy	0.476	0.390	0.433	0.434	0.761	2.494
6	Dist_To_CH	0.000	0.000	0.000	0.853	0.615	1.468
7	dist_CH_To_BS	0.055	0.535	0.433	0.216	0.126	1.366
8	Data_Forward_Ratio	0.750	0.102	0.219	0.195	0.063	1.329

Table 16: Top-8 features by SHAP importance

The most influential feature overall is **Time**, followed by **ADV_S** and **ADV_R**. The stacked colours show that different features drive different attacks, matching the networking intuition of Section 7.1: SCH_S matters almost only for TDMA, Data_Forward_Ratio and DATA_R matter mostly for Blackhole, and ADV_S produces the largest individual pushes towards Flooding (see the beeswarm below).

Caution: the simulation clock *Time* is also ranked highly. In WSN-DS, attacks are active in particular periods of each simulation run, so Time partly encodes the scenario rather than node behaviour. This is a dataset artifact that SHAP made visible; a production IDS should replace it with per-round or relative-time features (see Future Work).

13.3 Feature Impact per Attack (Beeswarm)

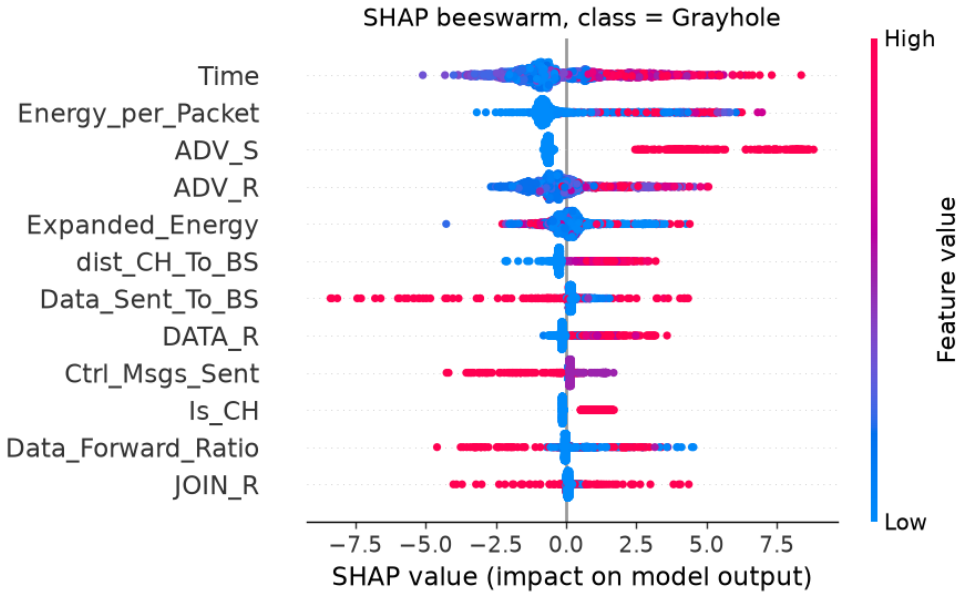


Figure 15: SHAP beeswarm for the Grayhole class (red = high feature value)

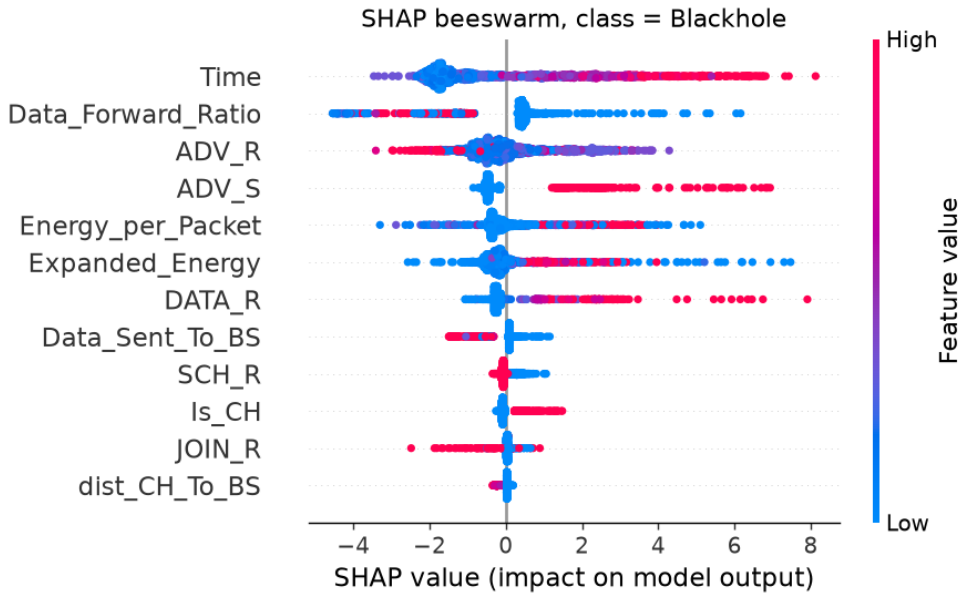


Figure 16: SHAP beeswarm for the Blackhole class

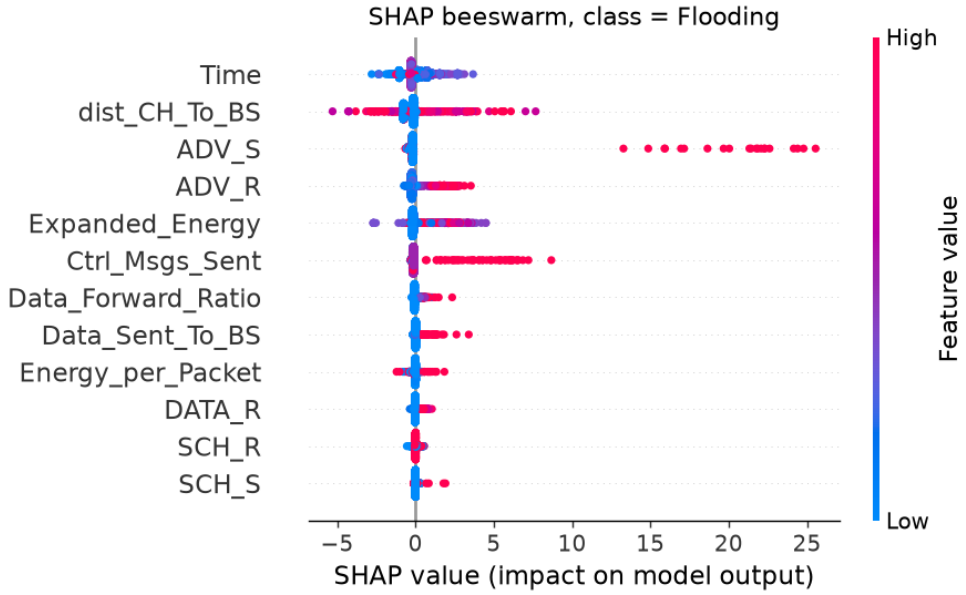


Figure 17: SHAP beeswarm for the Flooding class

Reading guide: each dot is one node; its x-position is how much the feature pushed the prediction towards the class, its colour is the feature value. For Flooding, high ADV_S values (red) push strongly to the right, i.e. a node sending many advertisement messages is a flooding attacker. For Blackhole/Grayhole, a high amount of received data combined with a low forwarding ratio pushes the prediction towards the attack.

13.4 Individual Prediction Explanation

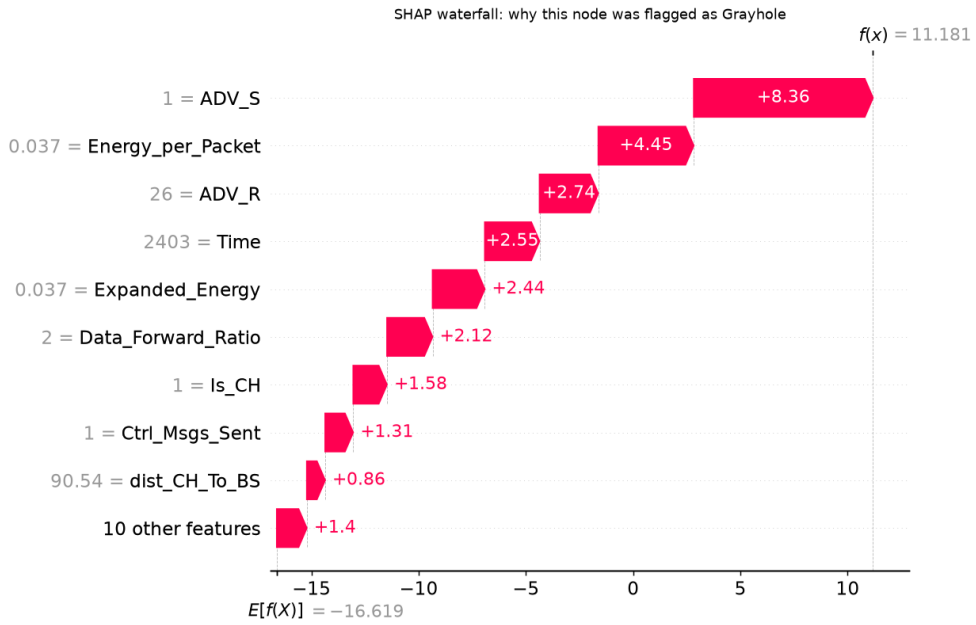


Figure 18: SHAP waterfall plot for one node correctly detected as Grayhole

The waterfall plot starts from the model's average output (base value) and adds the contribution of each feature for one specific node, ending at the final score for Grayhole. An operator can read exactly which counters (e.g. packets received vs forwarded) made the node suspicious.

13.5 LIME Local Explanation

LIME (Local Interpretable Model-agnostic Explanations) fits a simple linear model around one prediction. It gives human-readable rules such as value ranges of features that support or oppose the predicted class.

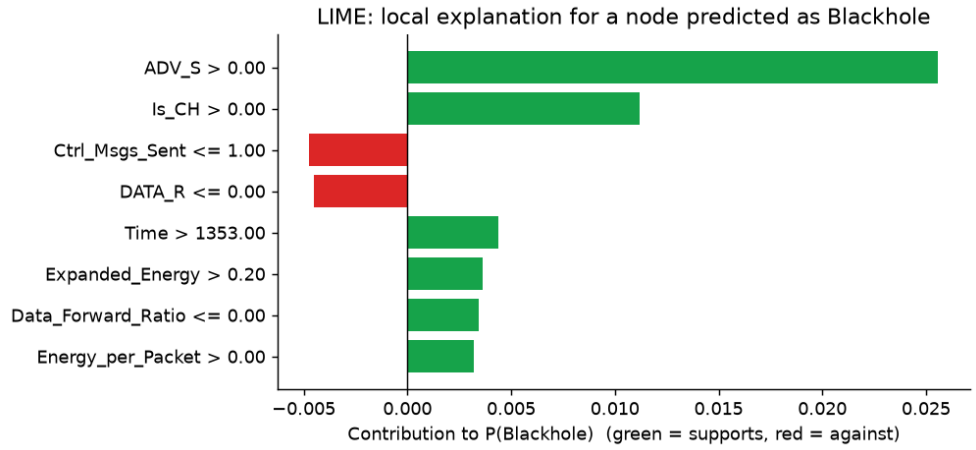


Figure 19: LIME explanation for a node predicted as Blackhole

SHAP and LIME agree on the key evidence, which increases trust in the model's decisions.

14 Implementation and Reproducibility

Item	Details
Language / tools	Python 3.13, scikit-learn, imbalanced-learn, XGBoost, LightGBM, SHAP, LIME, pandas, matplotlib, seaborn, reportlab
Hardware	Ordinary laptop CPU (no GPU needed); full pipeline runs in about 13 minutes
Disk space	About 30 MB dataset + 10 MB figures/results
Dataset	WSN-DS CSV (374,661 x 19) placed in data/WSN-DS.csv
Scripts	wsn_ids.py (full pipeline), leakage_check.py (leakage test), balancing_study.py (balancing ablation), make_methodology.py (Figure 3), build_report.py (this PDF)
Code	https://github.com/RHRONY05/WSN-Malicious-Node-Detection

Table 17: Implementation environment

PYTHON | Listing 10: Commands to reproduce all results

```
pip install scikit-learn imbalanced-learn xgboost lightgbm shap lime pandas seaborn r
eportlab
python wsn_ids.py           # trains + evaluates, writes figures/ and results/
python leakage_check.py     # leakage experiment
python balancing_study.py   # balancing-strategy ablation
python make_methodology.py  # methodology figure
python build_report.py      # builds this PDF report
```

15 Conclusion and Future Work

This project developed an explainable machine-learning intrusion detection system that identifies malicious nodes in LEACH-based wireless sensor networks. The main findings are:

- **High and honest performance:** 99.70% accuracy, 97.91% macro-F1 and MCC 0.9797 on an untouched, imbalanced test set after removing identifiers and duplicates.
- **Better rare-attack detection:** macro-F1 improved by +0.46 pp over the best baseline through class weighting, tuning, ensembling and class-threshold optimisation.
- **Leakage awareness:** we showed that keeping duplicates inflates scores and that models latch onto node identifiers, pitfalls that published WSN-DS studies rarely address.
- **Explainability:** SHAP and LIME confirm the model relies on meaningful routing behaviour (advertisement flooding, schedule messages, received-vs-forwarded data, energy use), not on artefacts.

Limitations and future work: WSN-DS is simulated (NS-2, LEACH only); validating on other protocols and real testbeds is needed. SHAP showed that the simulation *Time* feature carries scenario information, so a time-independent model should be evaluated. Future work includes a lightweight model using only the top-ranked features for on-node deployment, temporal models across consecutive rounds to separate Grayhole from Normal, and federated learning across cluster heads.

References

[1] I. Almomani, B. Al-Kasasbeh and M. Al-Akhras, "WSN-DS: A Dataset for Intrusion Detection Systems in Wireless Sensor Networks," *Journal of Sensors*, vol. 2016, Article ID 4731953, 2016. doi:10.1155/2016/4731953

[2] M. A. Talukder, S. Sharmin, M. A. Uddin, M. M. Islam and S. Aryal, "MLSTL-WSN: Machine Learning-based Intrusion Detection using SMOTETomek in WSNs," *International Journal of Information*

Security, Springer, 2024. doi:10.1007/s10207-024-00833-z (arXiv:2402.13277)

[3] S. Ouhmi, H. Khalid, M. Marwan, H. Silkhi and A. Ait Temghart, "Improved boosting-based machine learning algorithms for network intrusion detection in wireless sensor network," *IAES International Journal of Artificial Intelligence (IJ-AI)*, vol. 15, no. 3, pp. 2278-2289, 2026.

[4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *NeurIPS*, 2017.

[5] M. T. Ribeiro, S. Singh and C. Guestrin, "Why Should I Trust You?: Explaining the Predictions of Any Classifier," *ACM SIGKDD*, 2016.

[6] N. V. Chawla et al., "SMOTE: Synthetic Minority Over-sampling Technique," *JAIR*, vol. 16, 2002.

[7] W. B. Heinzelman, A. P. Chandrakasan and H. Balakrishnan, "An application-specific protocol architecture for wireless microsensor networks (LEACH)," *IEEE Trans. Wireless Communications*, 2002.